
Z.AI · DESKTOP AUTOMATION

Z.AGENT

Agent de bureau autonome

Pilotez votre ordinateur à distance via Telegram.

Transformez z.ai coding plan en assistant de bureau 100% autonome.

Version 1.0.0 · Juin 2026

Propulsé par GLM-4.6, GLM-4V, GLM-4.5 (et GLM-5.x quand disponible)

Table des matières

1. Introduction et vue d'ensemble	4
2. Architecture du système	6
3. Installation et configuration	9
4. Modules fonctionnels	13
4.1. Screen Control — curseur et VLM	13
4.2. File Manager — gestion des fichiers	14
4.3. Email Client — IMAP et SMTP	15
4.4. Calendar — calendrier ICS	16
4.5. Browser Control — Playwright	17
4.6. System Control — processus et notifications	18
5. Interface Telegram	19
6. Dashboard Web	22
7. Sécurité et bonnes pratiques	24
8. Exemples d'utilisation	26
9. Dépannage et FAQ	29
Annexe A — Référence des actions	31

1. Introduction et vue d'ensemble

1.1 Qu'est-ce que Z.AGENT ?

Z.AGENT est un agent de bureau autonome propulsé par les modèles GLM de z.ai. Il transforme votre ordinateur en un système pilotable à distance : depuis votre smartphone via Telegram, ou depuis un tableau de bord web local, vous lui confiez des tâches en langage naturel et il les exécute automatiquement. L'agent est capable de déplacer le curseur, cliquer, taper du texte, gérer vos fichiers, lire et envoyer des emails, consulter votre calendrier, piloter un navigateur web, et contrôler les processus système. Il fonctionne en autonomie complète, sans supervision humaine nécessaire, ce qui le rend adapté à des scénarios comme la gestion de votre poste de travail pendant vos absences, l'automatisation de tâches répétitives, ou l'exécution de requêtes à distance quand vous n'avez pas accès à votre machine.

La philosophie du projet est simple : transformer le « z.ai coding plan » en un véritable agent de bureau. Plutôt que de limiter l'IA à générer du code ou répondre à des questions, Z.AGENT lui donne le contrôle physique de votre ordinateur. Vous pouvez être en déplacement, en réunion, ou simplement absent — l'agent reste actif, écoute vos instructions Telegram, planifie les actions nécessaires, et les exécute en utilisant les bons modules pour chaque type de tâche. Le système repose sur une architecture multi-modèles où chaque GLM joue un rôle spécialisé : la planification, la perception visuelle, et l'exécution rapide.

1.2 Cas d'usage typiques

Organisation automatique

L'agent surveille votre dossier Téléchargements et le trie par type de fichier (images, documents, archives, code, vidéos). Il peut aussi archiver les fichiers anciens et nettoyer les doublons.

Gestion des emails

Pendant vos absences, l'agent lit vos emails non lus, vous envoie un résumé via Telegram, flagge les urgents, et peut répondre automatiquement aux messages simples avec votre accord.

Préparation de réunions

L'agent consulte votre calendrier, prépare l'agenda de la journée, ouvre les documents pertinents dans les bonnes applications, et envoie des rappels aux participants.

Automatisation browser

L'agent ouvre des sites web, remplit des formulaires, extrait des données, et scrape des pages selon vos besoins. Idéal pour surveiller des prix, remplir des déclarations, ou collecter des informations.

Maintenance système

L'agent surveille les processus en cours, tue ceux qui consomment trop de ressources, lance les applications dont vous avez besoin, et vous notifie en cas de problème.

Tâches programmées

Combinez l'agent avec le planificateur intégré pour exécuter des tâches récurrentes : sauvegardes, nettoyages, vérifications, ou tout autre scénario automatisable.

1.3 Modèles z.ai utilisés

Z.AGENT utilise une architecture multi-modèles pour optimiser les performances et le coût. Chaque modèle GLM joue un rôle précis dans la chaîne de traitement, en fonction de ses forces. Cette approche modulaire permet d'utiliser le bon modèle pour la bonne tâche, évitant ainsi de surconsommer des tokens sur des actions simples qui ne nécessitent pas un raisonnement complexe.

Rôle	Modèle	Usage
Planificateur	GLM-4.6	Décomposition des requêtes en plans d'actions multi-étapes. Raisonnement complexe.
Perception VLM	GLM-4V	Analyse des captures d'écran. Localisation des éléments UI. Compréhension visuelle.
Exécuteur	GLM-4.5	Actions simples et rapides. Parsing de réponses. Formatage de résultats.
Futur	GLM-5.1 / 5.2	Lorsque disponibles, remplaceront GLM-4.6 en tant que planificateur principal.

Pour activer GLM-5.x lorsque ces modèles seront disponibles, éditez le fichier `config/config.yaml` et changez la valeur de `zai.models.planner`. Le routeur multi-modèles s'occupe du reste.

2. Architecture du système

2.1 Vue d'ensemble

L'architecture de Z.AGENT est organisée en couches clairement séparées. Au sommet, les interfaces utilisateur (Telegram, dashboard web, CLI) reçoivent les requêtes et les transmettent à l'agent. L'agent orchestre ensuite trois composants spécialisés : le planificateur qui décompose la tâche, l'exécuteur qui lance les actions, et la mémoire qui persiste le contexte. Les actions sont finalement routées vers les modules fonctionnels (screen, files, email, calendar, browser, system) qui interagissent avec le système hôte.

2.2 Composants principaux

Agent Loop

L'agent loop est la boucle principale qui tourne en continu en arrière-plan. Elle écoute la file d'attente des tâches, dépile les requêtes dans l'ordre, et les traite une par une via le pipeline planification → exécution. Entre chaque tâche, elle effectue un ménage périodique : suppression des anciennes captures d'écran, vérification des rappels de calendrier, et exécution des tâches planifiées. L'agent expose son état (idle, planning, executing, paused, stopped) qui est visible en temps réel sur le dashboard et notifié via Telegram.

Planner (Planificateur)

Le planner prend une requête en langage naturel (ex : « Trie mes téléchargements par type ») et la décompose en une séquence d'actions atomiques exécutables. Il utilise un prompt système détaillé qui liste toutes les actions disponibles, leur signature, et les règles de sécurité. La sortie est un objet JSON structuré contenant la compréhension de la requête, le plan complet, les risques identifiés, et une estimation du temps nécessaire. En cas d'échec d'une étape, le planner peut être rappelé pour générer un plan alternatif à partir de l'état courant.

Executor (Exécuteur)

L'executor reçoit le plan généré et exécute chaque étape séquentiellement. Il route chaque action vers le handler de module approprié, applique les vérifications de sécurité avant l'exécution, et journalise le résultat (succès, erreur, durée). Un délai de sécurité configurable sépare chaque action pour éviter les effets de bord liés à la rapidité d'exécution. L'executor supporte aussi les handlers synchrones et asynchrones indifféremment.

Perception VLM

Le module de perception utilise GLM-4V pour comprendre visuellement l'écran. Il capture des screenshots à la demande, les redimensionne pour optimiser le coût des tokens, et les envoie au modèle de vision avec une question spécifique. Il peut localiser un élément UI par description (« le bouton Envoyer en bas à droite »), décrire le contenu global de l'écran, ou vérifier qu'une action a eu l'effet attendu. Les coordonnées renvoyées par le VLM sont recalculées en résolution native avant d'être passées à PyAutoGUI.

Memory (Mémoire)

La mémoire est divisée en deux tiers : volatile (session) et persistante (fichier JSON). La mémoire volatile stocke le contexte de la session en cours (états transitoires, variables de travail). La mémoire persistante conserve les faits utilisateur (préférences, raccourcis appris, historique des tâches) entre les redémarrages. Le module peut rechercher dans l'historique par mots-clés, ce qui permet à l'agent de retrouver le contexte d'une tâche similaire déjà exécutée.

2.3 Flux d'exécution d'une tâche

Le parcours complet d'une tâche, depuis l'instruction utilisateur jusqu'au résultat final, suit ces étapes :

Étape	Composant	Action
1	Interface	L'utilisateur envoie une requête via Telegram, le dashboard, ou la CLI.
2	Agent Loop	La tâche est ajoutée à la file d'attente avec un ID unique.
3	Planner	GLM-4.6 décompose la requête en plan d'actions atomiques (JSON).
4	Executor	Chaque étape est exécutée par le module approprié après vérification de sécurité.
5	Perception	Si l'action nécessite un retour visuel, GLM-4V analyse l'écran pour confirmer.
6	Memory	La tâche et son résultat sont enregistrés dans l'historique persistant.
7	Interface	Le résultat est notifié à l'utilisateur (Telegram / dashboard en temps réel).

3. Installation et configuration

3.1 Prérequis système

Z.AGENT fonctionne sur les trois systèmes d'exploitation majeurs : Windows 10/11, macOS 11+, et Linux (Ubuntu 20.04+ ou équivalent). Il nécessite Python 3.10 ou supérieur, ainsi que les droits d'administration pour l'installation des dépendances système. Sur Linux, vous devrez installer les bibliothèques X11 pour le contrôle écran. Sur macOS, vous devrez accorder les permissions d'accessibilité à Python dans Préférences Système → Sécurité et confidentialité → Confidentialité → Accessibilité. Sur Windows, le contrôle écran fonctionne nativement sans configuration supplémentaire.

3.2 Installation des dépendances

Clonez ou téléchargez le projet, puis installez les dépendances Python :

```
cd desktop-agent
python -m venv venv
source venv/bin/activate # Linux/macOS
# ou: venv\Scripts\activate # Windows
pip install -r requirements.txt
playwright install chromium
```

3.3 Obtention des clés API

Clé API z.ai

Rendez-vous sur <https://z.ai/> et créez un compte. Une fois connecté, accédez au tableau de bord développeur et générez une clé API. Cette clé est nécessaire pour appeler les modèles GLM (4.5, 4.6, 4V). Conservez-la précieusement, elle sera utilisée pour toutes les requêtes IA. Le coût est facturé à l'usage, mais reste très abordable pour un usage personnel — comptez environ 0,01 € par tâche complexe en moyenne.

Token Telegram Bot

Pour contrôler l'agent à distance via Telegram, créez un bot en parlant à @BotFather sur Telegram. Envoyez la commande /newbot, suivez les instructions pour nommer votre bot, et récupérez le token d'API au format numérique. Ensuite, récupérez votre identifiant utilisateur Telegram en parlant à @userinfobot — cet ID sera utilisé pour restreindre l'accès à votre bot (personne d'autre que vous ne pourra donner des ordres à votre agent).

Identifiants email (optionnel)

Pour activer le module email, vous aurez besoin d'un mot de passe d'application (et non de votre mot de passe habituel). Pour Gmail, activez la validation en deux étapes sur votre compte, puis générez un mot de passe d'application sur <https://myaccount.google.com/apppasswords>. Pour Outlook et les autres fournisseurs, la procédure est similaire. Ce mot de passe sera stocké dans la configuration et utilisé pour les connexions IMAP et SMTP.

3.4 Fichier de configuration

Toutes les variables sensibles doivent être placées dans un fichier `.env` à la racine du projet, qui ne sera jamais versionné :

```
# .env
ZAI_API_KEY=votre-cle-api-z.ai
TELEGRAM_BOT_TOKEN=123456:ABC-DEF...
EMAIL_USER=vous@gmail.com
EMAIL_APP_PASSWORD=aaaa-bbbb-cccc-dddd
```

Le fichier `config/config.yaml` contient tous les autres paramètres : dossiers surveillés, règles d'organisation, modèles IA à utiliser, seuils de confiance VLM, intervalles de perception, et politique de sécurité. Chaque section est commentée en détail. Vous pouvez surcharger n'importe quelle valeur via une variable d'environnement avec le préfixe `ZDA_` (ex : `ZDA_screen_scale=0.5`).

3.5 Vérification de l'installation

Avant le premier lancement, exécutez la commande de vérification pour vous assurer que tout est en place :

```
python main.py --check
```

Cette commande vérifie la présence du fichier de configuration, l'existence des clés API, et l'installation correcte de chaque dépendance Python. Elle affiche un rapport clair avec des verts pour les éléments validés et des oranges pour les éléments manquants. Une fois tous les checks au vert, vous pouvez lancer l'agent.

3.6 Premier lancement

Le mode serveur (par défaut) démarre simultanément l'agent, le bot Telegram, l'API web FastAPI sur le port 8765, et le planificateur :

```
python main.py
```

Pour tester rapidement sans Telegram ni dashboard, utilisez le mode CLI interactif : vous saisissez vos demandes directement dans le terminal et voyez le résultat. Pour exécuter une tâche unique et sortir, utilisez `--task "votre demande"`. Le mode CLI est idéal pour déboguer un module spécifique ou valider que l'agent comprend correctement vos requêtes.

4. Modules fonctionnels

Z.AGENT est composé de six modules spécialisés, chacun responsable d'un domaine fonctionnel. Chaque module enregistre ses actions auprès de l'exécuteur central, qui route les requêtes du planner vers le bon handler. Cette architecture modulaire permet d'ajouter, retirer, ou remplacer un module sans affecter le reste du système.

4.1 Screen Control — curseur et VLM

Le module Screen Control est le cœur du pilotage visuel. Il s'appuie sur PyAutoGUI pour le contrôle bas niveau (clics, frappe, défilement) et sur la perception VLM pour la compréhension de l'interface. Il peut cliquer sur un élément décrit en langage naturel (« le bouton bleu en haut à droite »), taper du texte y compris en Unicode via le presse-papier pour contourner les limitations de PyAutoGUI, faire des raccourcis clavier, et faire des drag-and-drop. Le module inclut aussi un mécanisme de retry : si un élément n'est pas trouvé du premier coup, il retente la capture et l'analyse.

Actions disponibles : `screen.click_element`, `screen.click_xy`, `screen.type_text`, `screen.press_key`, `screen.hotkey`, `screen.scroll`, `screen.screenshot`, `screen.wait`, `screen.find_and_click`, `screen.drag`.

4.2 File Manager — gestion des fichiers

Le module File Manager gère tout ce qui touche aux fichiers et dossiers. Il peut lister, déplacer, copier, renommer, supprimer (vers la corbeille par défaut), organiser automatiquement par type d'extension, rechercher par nom ou par contenu, et lire/écrire du texte. Les règles d'organisation sont entièrement configurables dans le fichier YAML : vous définissez les catégories (Images, Documents, Archives, Code, etc.) et les extensions associées. L'agent peut aussi faire des dry-runs pour voir ce qui serait déplacé avant de commiter l'organisation.

Actions disponibles : `files.list`, `files.move`, `files.copy`, `files.rename`, `files.delete`, `files.organize`, `files.search`, `files.read`, `files.write`, `files.create_dir`.

4.3 Email Client — IMAP et SMTP

Le module Email gère la réception via IMAP et l'envoi via SMTP. Il supporte les connexions SSL/TLS, les pièces jointes, et le HTML. Côté lecture, il peut récupérer les emails non lus, rechercher par sujet ou contenu, marquer comme lu, et lister les dossiers. Côté envoi, il gère les destinataires multiples (To, Cc, Bcc), les pièces jointes, et le format HTML. Le module fonctionne avec tous les fournisseurs standards : Gmail, Outlook, Yahoo, et tout serveur IMAP/SMTP personnalisé. La configuration se fait dans la section email du fichier `config.yaml`.

Actions disponibles : `email.send`, `email.read_unread`, `email.search`, `email.reply`, `email.mark_read`, `email.list_folders`.

4.4 Calendar — calendrier ICS

Le module Calendar gère les événements au format ICS, le standard universel des calendriers. Il peut lire les fichiers ICS exportés depuis Google Calendar, Outlook, ou Apple Calendar, et gère aussi un calendrier local que l'agent peut modifier. Il supporte les événements récurrents (via la bibliothèque `recurring-ical-events`), peut créer de nouveaux événements, en supprimer, en chercher par texte, et positionner des rappels qui déclencheront une notification système et un message Telegram X minutes avant l'événement.

Actions disponibles : `calendar.list`, `calendar.create`, `calendar.delete`, `calendar.search`, `calendar.remind`.

4.5 Browser Control — Playwright

Le module Browser Control s'appuie sur Playwright pour piloter un navigateur Chromium, Firefox, ou WebKit. Il peut ouvrir des URLs, cliquer sur des sélecteurs CSS, remplir des formulaires, extraire du texte ou du HTML, prendre des screenshots, scroller, et exécuter du JavaScript arbitraire sur la page. Le navigateur peut tourner en mode headless (invisible) ou visible pour le débogage. Le profil utilisateur est persistant, ce qui permet de garder les sessions connectées entre deux lancements. Idéal pour les tâches de scraping, de remplissage de formulaires répétitifs, ou de surveillance de sites web.

Actions disponibles : `browser.open`, `browser.click`, `browser.fill`, `browser.screenshot`, `browser.extract`, `browser.scroll`, `browser.evaluate`, `browser.close`.

4.6 System Control — processus et notifications

Le module System Control gère les opérations système : lancement et arrêt d'applications, liste des processus avec leur consommation CPU/RAM, notifications système cross-platform, presse-papier, ouverture de fichiers dans leur application par défaut, et exécution de commandes shell (avec filtrage des commandes dangereuses). Le lancement d'applications est par défaut restreint à une whitelist configurable, ce qui empêche l'agent de lancer des applications arbitraires sans votre accord préalable. Le module inclut aussi une commande d'informations système qui remonte CPU, RAM, disque, et version de l'OS.

Actions disponibles : `system.launch_app`, `system.kill_app`, `system.list_processes`, `system.notification`, `system.clipboard_get`, `system.clipboard_set`, `system.open_path`, `system.system_info`, `system.run_command`.

5. Interface Telegram

5.1 Configuration du bot

L'interface Telegram est le canal principal pour piloter l'agent à distance. Après avoir créé votre bot via @BotFather et récupéré le token, ajoutez-le dans le fichier .env. Important : définissez votre identifiant utilisateur dans le champ `allowed_user_ids` de la configuration Telegram. Cela restreint l'usage du bot à vous seul, empêchant quiconque d'autre de donner des ordres à votre agent. Si cet identifiant est à 0, le bot répond à tout le monde (déconseillé en production).

5.2 Commandes slash disponibles

Les commandes slash offrent un accès rapide aux fonctionnalités principales. Elles sont prévisibles, scriptables, et évitent d'utiliser des tokens IA pour des tâches simples. Pour les tâches complexes, utilisez plutôt le mode langage naturel.

Commande	Description
/start	Vérifier le statut de l'agent et confirmer qu'il est en ligne.
/status	État détaillé : tâche courante, file d'attente, mémoire.
/help	Afficher l'aide complète avec toutes les commandes.
/screenshot	Capture d'écran instantanée envoyée directement dans Telegram.
/pause /resume	Mettre l'agent en pause ou le reprendre.
/cancel	Vider la file d'attente des tâches en attente.
/memory	Afficher l'état de la mémoire (faits, préférences, raccourcis).
/files organize	Trier le dossier Téléchargements par type de fichier.
/files list [path]	Lister les fichiers d'un dossier (Bureau par défaut).
/email unread	Lire les 5 derniers emails non lus avec résumé.
/email send to subject body	Envoyer un email (séparez les champs par).
/calendar list	Lister les 10 prochains événements de calendrier.
/system info	Afficher les informations système (CPU, RAM, disque).
/browser open url	Ouvrir une URL dans le navigateur.

5.3 Mode langage naturel

Au-delà des commandes slash, vous pouvez simplement écrire votre demande en langage naturel. L'agent utilise GLM-4.6 pour comprendre l'intention, planifier les actions nécessaires, et les exécuter. Ce mode est plus souple mais consomme des tokens IA. Quelques exemples :

- « Tri mes téléchargements par type de fichier »
- « Envoie un mail à alice@example.com pour confirmer la réunion de demain à 14h »
- « Capture l'écran et dis-moi ce qu'il y a dessus »
- « Ouvre Gmail dans le navigateur et connecte-toi »
- « Liste mes 10 prochains événements de calendrier »
- « Trouve tous les fichiers PDF créés cette semaine dans Documents »
- « Ferme toutes les fenêtres Chrome et relance-les »
- « Prépare un email avec en pièce jointe le dernier fichier du Bureau »

Quand vous envoyez une demande, l'agent confirme la réception avec un ID de tâche, puis vous notifie du résultat une fois la tâche terminée. Si une étape échoue, le message indique quelle étape et quelle erreur, ce qui vous permet d'ajuster votre demande ou de relancer.

6. Dashboard Web

6.1 Vue d'ensemble

Le dashboard web est une interface Next.js qui se connecte à l'API FastAPI de l'agent pour offrir une visualisation en temps réel de son activité. Il affiche l'état courant (idle, planning, executing, paused), la file d'attente, la mémoire, les logs en streaming, les captures d'écran, et l'historique des tâches. C'est l'outil idéal pour surveiller l'agent quand vous êtes devant votre ordinateur, ou pour déboguer une tâche qui ne se comporte pas comme prévu.

6.2 Installation du dashboard

Le dashboard est un projet Next.js séparé. Pour le lancer :

```
cd dashboard
bun install
bun run dev
```

Le dashboard écoute par défaut sur le port 3000 et se connecte à l'API agent sur le port 8765. Si votre API agent est sur une autre machine, définissez la variable d'environnement `NEXT_PUBLIC_AGENT_API` avec l'URL complète (ex : `http://192.168.1.10:8765`). Le dashboard gère automatiquement la reconnexion WebSocket en cas de coupure réseau.

6.3 Fonctionnalités principales

Statut temps réel

L'état de l'agent est mis à jour en direct via WebSocket. Vous voyez immédiatement quand il passe de idle à planning, puis à executing.

Soumission de tâches

Un formulaire permet de saisir des tâches directement depuis le dashboard, avec des boutons de raccourcis pour les tâches les plus courantes (trier fichiers, lire emails, etc.).

Historique des tâches

Chaque tâche exécutée est listée avec son plan détaillé (chaque étape, son action, son résultat). Cliquez sur une tâche pour voir le déroulement complet.

Logs en streaming

Les logs de l'agent sont diffusés en temps réel via WebSocket, avec code couleur par niveau (INFO, WARNING, ERROR). Idéal pour déboguer en direct.

Galerie de captures

Toutes les captures d'écran prises par le module VLM sont visibles dans une galerie, avec timestamp. Vous pouvez cliquer pour voir en grand.

Contrôle de l'agent

Boutons Pause / Resume / Stop pour contrôler l'agent directement depuis le dashboard, sans passer par Telegram.

6.4 Architecture du dashboard

Le dashboard est construit avec Next.js 16, React 19, TypeScript, Tailwind CSS 4, et shadcn/ui. Il utilise deux WebSockets : un pour les logs (ws/logs) et un pour les événements de progression (ws/progress). L'API REST est utilisée pour les actions ponctuelles (soumission de tâche, commande de contrôle, récupération des screenshots). Le polling de statut toutes les 3 secondes sert de fallback si la WebSocket se déconnecte. Le design est en thème sombre avec accents emerald et teal, optimisé pour une lecture prolongée sans fatigue oculaire.

7. Sécurité et bonnes pratiques

7.1 Politique de sécurité par défaut

Z.AGENT est configuré par défaut en mode « plein contrôle » : l'agent peut exécuter des actions destructives sans demander de confirmation. Ce mode est adapté à un usage de confiance où vous êtes seul à utiliser l'agent et où vous acceptez les risques. Toutefois, même en plein contrôle, certaines protections restent actives en permanence pour éviter les catastrophes.

Chemins protégés : les dossiers `~/ssh`, `~/aws`, `~/.config/1password`, `/etc/passwd`, `/etc/shadow` ne sont jamais accessibles en lecture, écriture, ou suppression. Cette liste est configurable dans la section `security.protected_paths` du fichier de configuration. Ajoutez-y tout dossier sensible propre à votre environnement (coffres-forts cryptographiques, dossiers de configuration de mots de passe, etc.).

Actions bloquées : certaines actions sont interdites par défaut, quel que soit le mode. Il s'agit de `format_disk`, `rm_rf_root`, `modify_system_files`, `shutdown_system`, `reboot_system`. L'agent refusera toujours de les exécuter, même si vous lui demandez explicitement. Cette liste est aussi configurable, mais nous recommandons de la laisser telle quelle.

7.2 Bonnes pratiques recommandées

Restreindre l'accès Telegram

Configurez `allowed_user_ids` avec votre identifiant Telegram. Sans cette restriction, n'importe qui connaissant le nom de votre bot pourrait lui envoyer des ordres.

Utiliser un mot de passe d'application

Ne mettez jamais votre mot de passe email principal dans la configuration. Utilisez toujours un mot de passe d'application dédié, révocable à tout moment.

Sauvegarder régulièrement

L'agent peut supprimer des fichiers. Activez Time Machine (macOS), File History (Windows), ou un équivalent Linux. Testez vos sauvegardes périodiquement.

Limiter la whitelist d'apps

Ne mettez dans `system.allowed_apps` que les applications que l'agent a réellement besoin de lancer. Plus la liste est courte, plus la surface d'attaque est réduite.

Surveiller les logs

Consultez régulièrement les logs du dashboard. Si vous voyez des actions inattendues, mettez l'agent en pause et investiguez.

Tester en dry-run

Pour les tâches sensibles (organisation de fichiers, suppressions), utilisez le paramètre `dry_run` quand il existe, afin de voir ce qui serait fait sans le faire réellement.

Mettre à jour régulièrement

Les dépendances Python et npm doivent être mises à jour périodiquement pour profiter des correctifs de sécurité. Exécutez `pip-audit` et `bun audit`.

7.3 Limites à connaître

Z.AGENT est puissant, mais a des limites qu'il faut garder à l'esprit. Premièrement, le module VLM peut se tromper dans la localisation des éléments UI, particulièrement sur des interfaces complexes ou non standard. Si la confiance est inférieure au seuil (0.7 par défaut), le clic est refusé — augmentez le seuil pour plus de prudence, diminuez-le pour plus de permissivité. Deuxièmement, l'agent ne sait pas gérer les captchas ni les authentifications à deux facteurs — il vous faudra pré-authentifier les sessions dans le profil browser persistant. Troisièmement, certaines applications ont des protections anti-automation (DRM, détection de PyAutoGUI) qui peuvent faire échouer le contrôle écran.

8. Exemples d'utilisation

8.1 Organiser le dossier Téléchargements

Un des cas d'usage les plus simples et les plus utiles. Envoyez à l'agent :

```
"Tri mon dossier Téléchargements par type de fichier"
```

L'agent va : (1) lister le contenu du dossier ~/Downloads, (2) créer des sous-dossiers par catégorie (Images, Documents, Archives, Code, Vidéos, Audio), (3) déplacer chaque fichier dans le bon sous-dossier, (4) vous envoyer un résumé du nombre de fichiers déplacés par catégorie. Les fichiers sans extension reconnue restent à la racine.

8.2 Lire et résumer les emails non lus

Quand vous êtes en déplacement, demandez à l'agent :

```
"Lis mes 5 derniers emails non lus et fais-moi un résumé pour chacun"
```

L'agent va : (1) se connecter en IMAP à votre boîte, (2) récupérer les 5 derniers messages non lus, (3) pour chaque email extraire l'expéditeur, le sujet, et un résumé du corps via GLM-4.6, (4) vous envoyer le tout via Telegram. Vous pouvez ensuite demander à l'agent de répondre à l'un d'entre eux en utilisant email.reply.

8.3 Surveiller un site web

Pour vérifier régulièrement un prix ou une disponibilité :

```
"Ouvre https://example.com/product dans le navigateur, capture l'écran, et dis-moi quel est le prix affiché"
```

L'agent va : (1) lancer le navigateur Chromium via Playwright, (2) naviguer vers l'URL, (3) attendre le chargement de la page, (4) prendre une capture d'écran, (5) envoyer la capture à GLM-4V avec un prompt demandant le prix, (6) vous renvoyer la réponse. Vous pouvez combiner cette tâche avec le scheduler pour la répéter toutes les heures.

8.4 Préparer une réunion

Avant une réunion, demandez :

```
"Quand est ma prochaine réunion ? Ouvre les documents qui y sont liés et envoie-moi un résumé de l'agenda."
```

L'agent va : (1) consulter le calendrier ICS pour le prochain événement, (2) extraire le titre, la date, l'heure, le lieu, et la description, (3) chercher dans vos documents ceux dont le nom correspond au titre de la réunion, (4) ouvrir ces documents dans leur application par défaut, (5) vous envoyer un résumé sur Telegram avec l'agenda et les liens vers les documents ouverts.

8.5 Nettoyage système

Pour libérer de la mémoire ou tuer un processus récalcitrant :

```
"Liste les 20 processus qui consomment le plus de mémoire. Si Chrome utilise plus de 4 Go, ferme-le."
```

L'agent va : (1) lister les processus via psutil, (2) identifier les plus gourmands, (3) vérifier si Chrome dépasse 4 Go, (4) si oui, tuer tous les processus Chrome, (5) vous confirmer l'action effectuée. Cette tâche illustre la capacité de l'agent à faire des décisions conditionnelles basées sur l'état du système.

9. Dépannage et FAQ

9.1 L'agent ne répond pas sur Telegram

Vérifiez d'abord que le token Telegram est correctement défini dans le fichier `.env` et que l'agent tourne (vous devriez voir « Telegram interface started » dans les logs). Vérifiez ensuite que votre identifiant utilisateur est bien dans `allowed_user_ids`. Si le bot est en ligne mais ne répond pas, regardez les logs côté agent — une erreur de connexion à l'API `z.ai` est souvent la cause (clé expirée, quota dépassé, problème réseau).

9.2 L'agent clique au mauvais endroit

Le module VLM peut parfois mal localiser un élément UI. Augmentez le seuil de confiance `screen.click_confidence` à 0.8 ou 0.9 pour exiger une meilleure certitude. Réduisez aussi `screen.scale` à 0.5 — une résolution plus basse accélère le traitement VLM mais peut réduire la précision. À l'inverse, monter `scale` à 1.0 donne plus de détails au modèle. Pour les éléments critiques, utilisez `screen.find_and_click` qui retente automatiquement la recherche si le premier essai échoue.

9.3 L'envoi d'email échoue

La cause la plus fréquente est l'utilisation du mot de passe principal au lieu d'un mot de passe d'application. Pour Gmail, activez la 2FA puis générez un app password sur myaccount.google.com/apppasswords. Vérifiez aussi que `imap_ssl` et `smtp_tls` sont à `true` dans la configuration. Pour Outlook, utilisez `imap-mail.outlook.com:993` et `smtp-mail.outlook.com:587`. Pour Yahoo, utilisez `imap.mail.yahoo.com:993` et `smtp.mail.yahoo.com:587` avec un app password.

9.4 Le navigateur Playwright ne se lance pas

Après installation des dépendances Python, vous devez aussi installer les navigateurs Playwright avec la commande `playwright install chromium`. Sans cette étape, le module `browser_control` échouera au premier appel. Sur Linux, vous aurez peut-être besoin de `playwright install-deps` pour installer les bibliothèques système (`libnss3`, `libatk-bridge2.0-0`, etc.).

9.5 Le dashboard ne se connecte pas à l'API

Vérifiez que l'API FastAPI tourne sur le port 8765 (vous devriez voir « Uvicorn running on http://127.0.0.1:8765 » dans les logs). Si l'API est sur une autre machine, définissez `NEXT_PUBLIC_AGENT_API`. Vérifiez les règles de firewall — le port 8765 doit être ouvert. Ouvrez la console du navigateur pour voir les erreurs WebSocket ou de fetch. Les erreurs CORS se résolvent en ajoutant votre origine dans `dashboard.cors_origins` du fichier de configuration.

9.6 L'agent consomme beaucoup de tokens IA

Chaque tâche consomme des tokens pour la planification (GLM-4.6) et éventuellement la perception (GLM-4V). Pour réduire la consommation : utilisez les commandes slash pour les tâches simples (elles contournent le planner), réduisez `max_actions_per_task` pour limiter les plans longs, et diminuez `screen.scale` pour réduire la taille des images envoyées au VLM. Vous pouvez aussi désactiver la perception VLM si vos tâches n'en ont pas besoin (utilisez plutôt des coordonnées fixes pour les clics répétitifs).

Annexe A — Référence des actions

Liste complète des actions disponibles pour le planner. Chaque action accepte les paramètres indiqués et retourne un dictionnaire avec au minimum une clé `success` (booléen).

Screen Control

Action	Paramètres
<code>screen.click_element</code>	<code>description: str, button: str = 'left', clicks: int = 1</code>
<code>screen.click_xy</code>	<code>x: int, y: int, button: str = 'left', clicks: int = 1</code>
<code>screen.type_text</code>	<code>text: str, interval: float = 0.0</code>
<code>screen.press_key</code>	<code>key: str, presses: int = 1</code>
<code>screen.hotkey</code>	<code>keys: List[str]</code>
<code>screen.scroll</code>	<code>direction: str, amount: int = 3, x: int?, y: int?</code>
<code>screen.screenshot</code>	<code>description: str?</code>
<code>screen.wait</code>	<code>seconds: float = 1.0</code>
<code>screen.find_and_click</code>	<code>description: str, max_retries: int = 2</code>
<code>screen.drag</code>	<code>x1, y1, x2, y2: int, duration: float = 0.5</code>

File Manager

Action	Paramètres
<code>files.list</code>	<code>path: str?, pattern: str = '*', include_hidden: bool = false</code>
<code>files.move</code>	<code>sources: List[str], destination: str</code>
<code>files.copy</code>	<code>sources: List[str], destination: str</code>
<code>files.rename</code>	<code>path: str, new_name: str</code>
<code>files.delete</code>	<code>path: str, permanent: bool = false</code>
<code>files.organize</code>	<code>path: str?, dry_run: bool = false</code>
<code>files.search</code>	<code>path: str?, pattern: str, content_query: str?</code>
<code>files.read</code>	<code>path: str, max_size: int = 1MB</code>
<code>files.write</code>	<code>path: str, content: str, append: bool = false</code>

files.create_dir	path: str
------------------	-----------

Email

Action	Paramètres
email.send	to: str, subject: str, body: str, cc: str?, bcc: str?, attachments: List[str]?, html: bool
email.read_unread	folder: str = 'INBOX', limit: int = 10, mark_seen: bool = false
email.search	query: str, folder: str = 'INBOX', limit: int = 20
email.reply	message_id: str, body: str, folder: str = 'INBOX'
email.mark_read	message_id: str, folder: str = 'INBOX'
email.list_folders	–

Calendar

Action	Paramètres
calendar.list	days_ahead: int = 7, include_recurring: bool = true
calendar.create	title: str, start: str, end: str?, description: str?, location: str?
calendar.delete	uid: str
calendar.search	query: str
calendar.remind	event_uid: str, minutes_before: int = 15

Browser

Action	Paramètres
browser.open	url: str, wait_until: str = 'domcontentloaded'
browser.click	selector: str, wait: bool = true
browser.fill	selector: str, value: str
browser.screenshot	full_page: bool = false
browser.extract	selector: str = 'body', attribute: str = 'text_content'
browser.scroll	direction: str = 'down', amount: int = 500

<code>browser.evaluate</code>	<code>script: str</code>
<code>browser.close</code>	–

System

Action	Paramètres
<code>system.launch_app</code>	<code>name: str, args: List[str]?</code>
<code>system.kill_app</code>	<code>name: str</code>
<code>system.list_processes</code>	<code>filter_name: str?, limit: int = 50</code>
<code>system.notification</code>	<code>title: str, message: str</code>
<code>system.clipboard_get</code>	–
<code>system.clipboard_set</code>	<code>content: str</code>
<code>system.open_path</code>	<code>path: str</code>
<code>system.system_info</code>	–
<code>system.run_command</code>	<code>command: str, cwd: str?, timeout: int = 60</code>